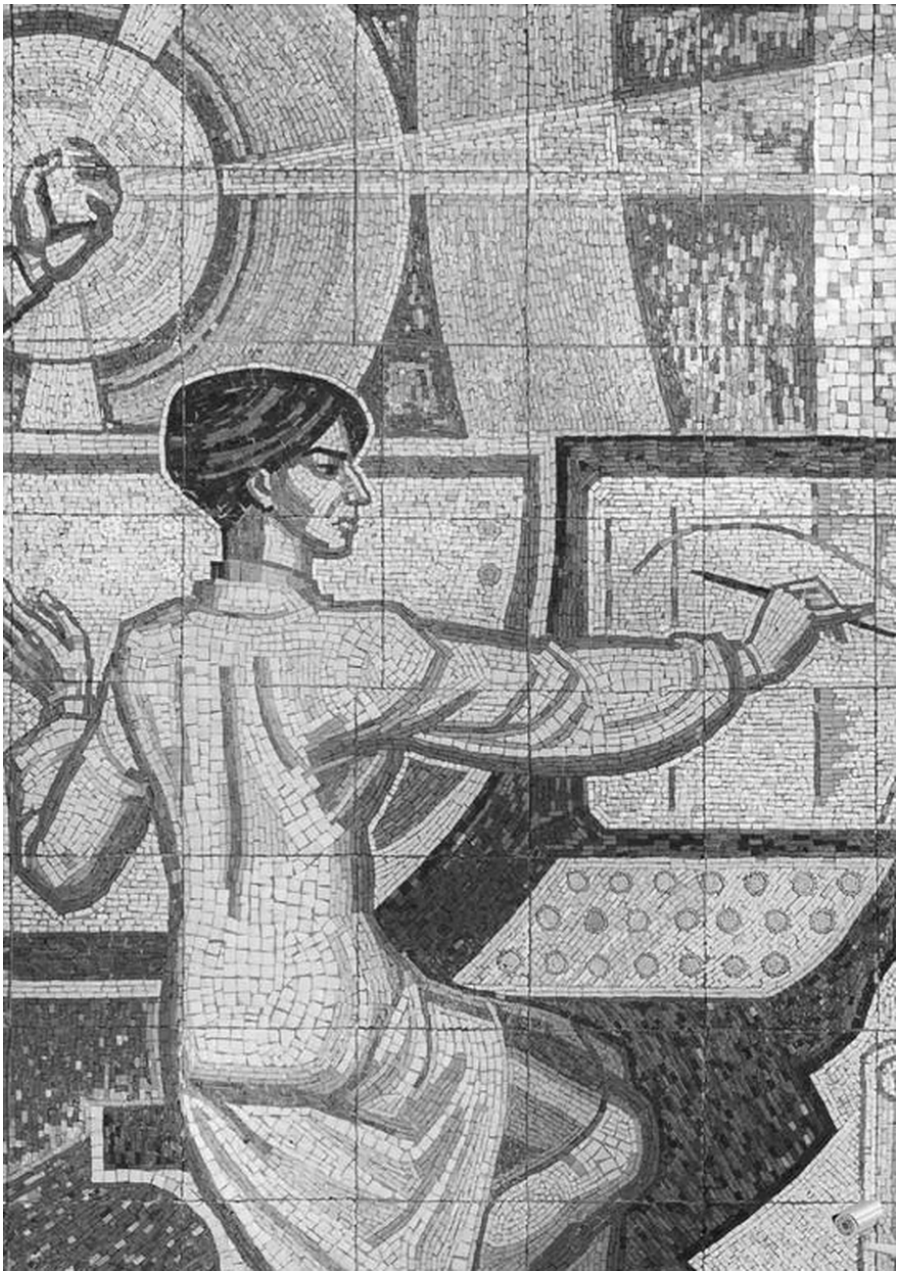


questions of cybernetics.

Issue 001

contents

Beginnings 4
On the RISC-V processor 5
Aldos from a bird's eye view 17
Benchmarking with Eratosthenes' sieve 27
Trifles 31
What a function equals 33
Conway's Game of Life 38
From programming language design to computer construction 44
On process isolation 64
Feedback 73
The guy from hell. Chapter 1 74

beginnings

ortfero

The present issue opens "Questions of Cybernetics", the bulletin of a temporary labor collective formed for the development of a general-purpose computer, the aldan-4. The work is conducted in deliberately limited scope. It addresses a circumstance now plainly visible: the systems on which contemporary computing rests have grown to a complexity that exceeds the grasp of any individual engineer.

The measure we have chosen is surveyability. Complexity has silted up layer on layer, and cannot be reduced by further addition; surveyability is approached only by fresh construction, with the benefit of accumulated experience.

Over several months the collective has designed the machine, written its emulator, and built upon it the operating system aldos, the procedural language almac, and the tooling that surrounds them. As this issue goes out, the system has reached working self-sufficiency in its local configuration: on the machine itself the full cycle of work is now possible with no recourse to any external means. It was this that moved us to begin publishing.

The bulletin will report the collective's work systematically; it will set out our design decisions and it will take up questions of a general theoretical character bearing on the system. We hope it becomes a place of useful exchange.

on the risc-v processor

pin47

abstract

The architecture of the 64-bit processor RISC-V is considered. Its base instruction set — the integer core with multiply-and-divide — is set out in summary tables and illustrated by a Euclid subroutine.

statement of the question

The choice of a base instruction-set architecture for the project is determined by three considerations: the regularity and small extent of the system of instructions, the openness of the specification, and the absence of accumulated historical compatibility.

Below, the RISC-V architecture is considered.

organization of the risc-v processor (rv64)

The base (integer) architecture RV64I has the following features.

— Word length. The word length of data and of the address is 64 bits (XLEN = 64). Operations on the byte (8 bits), the half-word (16), the word (32) and the double word (64) are supported.

— Register file. The processor contains 32 general-purpose registers of 64 bits each, designated

x0...x31. Register x0 is fixed at zero: reading yields zero, writing is ignored. This lets several operations (transfer, comparison with zero, unconditional transfer of control) be expressed uniformly without separate instructions. The address of the next instruction is held in a separate program counter, not counted among the general-purpose registers.

In the hardware, registers x1...x31 are interchangeable; the control unit assigns them no special purpose. Their roles are fixed not by the hardware but by an adopted calling convention, common to system and applied software. Observance of this convention ensures the interoperability of subroutines and the portability of programs between samples of the computer. The standard mnemonics and roles are given in table 1.

Table 1. Convention on the use of the general-purpose registers

Hardware	Mnemonic	Purpose	Preserved across a call
x0	zero	Constant zero	—
x1	ra	Subroutine return address	No (caller's concern)
x2	sp	Stack pointer	Yes (callee's concern)
x3	gp	Pointer to the global data area	— (unchanged)

Hardware	Mnemonic	Purpose	Preserved across a call
x4	tp	Pointer to task (thread) data	— (unchanged)
x5	t0	Temporary (also auxiliary link address)	No
x6-x7	t1-t2.	Temporary registers	No
x8	s0/fp	Saved; stack frame pointer.	Yes
x9	s1	Saved register	Yes
x10-x11	a0-a1	Subroutine arguments and returned values	No
x12-x17	a2-a7	Subroutine arguments	No
x18-x27	s2-s11	Saved registers	Yes
x28-x31	t3-t6	Temporary registers	No

— zero (x0) is the source of constant zero and the receiver of discarded results. ra (x1) holds the return address, written by the transfer-with-return instructions. sp (x2) addresses the top of the stack, on which lie the local data and saved registers of subroutines; fp (x8), where required, marks the start of the current stack frame. gp and tp (x3, x4) point to global program data and to task

(thread) data; they are not altered during execution.

- a0...a7 (x10...x17) pass arguments to a subroutine; the result is returned in a0, a1. s0...s11 are saved registers: a subroutine that uses them must restore the prior contents before returning, so the caller may rely on their invariance. t0...t6 are temporary: not preserved across a call, the caller must save them where needed. The division into saved and temporary registers reduces the number of memory accesses on subroutine calls.
- The "load-store" principle. The architecture is of the "load-store" type: memory is accessed only by load and store instructions; all arithmetic and logical instructions operate on register contents. This gives a regular pipeline and a small number of instruction formats.
- Instruction format. The instruction length is fixed at 32 bits (4 bytes). A small number of encoding formats differ only in the placement of register numbers and of the immediate constant: register-register (R), immediate (I), store (S), conditional branch (B), upper-bits immediate (U), unconditional transfer (J). Byte order in memory is little-endian.
- Execution modes. The specification provides machine, supervisor and user modes, which separate system and applied software and support protection under multitask operation. The aldan-4 implements machine mode alone: there is no paging and no memory protection, and every program sees one address space, all of it reachable. The reasons are taken up

in the correspondence on process isolation at the end of this issue.

— Extensibility. The integer core (I) may be supplemented by standard extensions: integer multiply and divide (M), atomic operations (A), floating-point (F, D), compressed instructions (C), and others. The base taken for the present project, and considered below, is RV64IM. What the machine has since added to it was in each case added against a measurement rather than in advance: the floating-point pair F and D; the bit-manipulation subsets Zba, Zbb and Zbs; a curated part of the vector extension V, with vector registers of 256 bits; the CSR and instruction-fence extensions Zicsr and Zifencei; and the debug triggers of Sdtrig, on which the debugger stands. Two extensions are left out on purpose. Without A there is no atomic operation, and none is needed: the machine has one hart, and masking interrupts is the only lock. Without C every instruction is 32 bits long, so code is read and patched at a fixed stride. None of the additions displaces the base, which remains what the rest of this article describes.

base instruction set (rv64im)

The base instruction set is orthogonal: operations, operands, and addressing modes combine uniformly. The principal classes are set out below.

— Arithmetic-logical register-register instructions. The result of an operation on two general-purpose

registers is placed in a third: addition add, subtraction sub, bitwise "and" and, "or" or, "exclusive or" xor, the logical and arithmetic shifts sll/srl/sra, comparisons with the setting of a flag slt/sltu.

- Instructions with an immediate operand. The same operations with one operand as a constant encoded in the instruction: addi, andi, ori, xori, slti/sltiu, shifts by a constant slli/srli/srai.
- Instructions for the formation of the upper bits. lui places a 20-bit constant into the upper bits of a register; auipc forms an address relative to the program counter. Together they provide loading of any 64-bit constant and position-independent addressing.
- "Word" instructions. For 32-bit operations there are instructions with the suffix "w" (addw, subw, sllw, srlw, sraw, addiw, and so on), operating on the lower 32 bits with sign-extension of the result to 64 bits.
- Instructions of memory access. Loading from memory into a register: lb, lh, lw, ld (with sign-extension), lbu, lhu, lwu (with zero-extension); storing from a register into memory: sb, sh, sw, sd.
- Instructions of the transfer of control. The unconditional transfer with preservation of the return address jal and the transfer by register jalr; conditional branches on the comparison of two registers: beq, bne, blt, bge, bltu, bgeu.

- System and synchronizing instructions. The supervisor call ecall, the breakpoint ebreak, the ordering of memory accesses fence. On the aldan-4, with one hart and devices that finish their work inside the store that asks for it, fence orders nothing and does nothing.
- The multiplication and division extension (M). Multiplication mul (the lower bits of the product), mulh/mulhsu/mulhu (the upper bits, for signed and unsigned operands), division div/divu, remainder rem/remu, and their "word" forms mulw, divw, divuw, remw, remuw.

In the summary below, the operand composition is indicated for each instruction in place of the formal class. Notation: rd — destination register; rs1, rs2 — source registers; imm — immediate operand (a constant encoded in the instruction); shamt — shift amount; off(rs1) — a memory address formed as rs1 + off; label — target address of a transfer.

Table 2. Summary of the base instructions of RV64IM

Mnemonic	Operands	Purpose
add, sub, and, or, xor, sll, srl, sra, slt, sltu	rd, rs1, rs2	Register-register arithmetic, logic, shifts, comparison
addw, subw, sllw, srlw, sraw	rd, rs1, rs2	The same upon the lower 32 bits ("word" operations)

Mnemonic	Operands	Purpose
mul, mulh, mulhsu, mulhu, div, divu, rem, remu	rd, rs1, rs2	Multiplication, division, remainder (M extension)
addi, andi, ori, xori, slti, sltiu	rd, rs1, imm	The same operations with an immediate operand
slli, srli, srai	rd, rs1, shamt	Shift by a constant amount
lui, auipc	rd, imm	Formation of the upper bits of a constant / of an address
lb, lh, lw, ld, lbu, lhu, lwu	rd, off(rs1)	Loading from memory into a register
sb, sh, sw, sd	rs2, off(rs1)	Storing from a register into memory
beq, bne, blt, bge, bltu, bgeu	rs1, rs2, label	Conditional branch upon a comparison of registers
jal	rd, label	Transfer keeping the return address
jalr	rd, off(rs1)	Transfer by register keeping the return address
ecall, ebreak	(none)	Call to the supervisor; breakpoint
fence	pred, succ	Ordering of the preceding and succeeding memory accesses

The register x0, fixed at zero, lets a number of common operations be expressed without a separate opcode: register transfer (addi with a zero constant), sign inversion (sub from x0), unconditional transfer (jal writing the address into x0), and so on. This economises the control unit while keeping the instruction set complete.

example of a program

As an example, consider a subroutine computing the greatest common divisor (gcd) of two non-negative integers by Euclid's algorithm. It uses the conditional branch, the remainder instruction of the M extension, register transfers, the transfer-with-return instructions, and the calling convention (arguments in a0, a1; result in a0).

The algorithm replaces the larger number by the remainder of the division until the second is zero; the gcd is the last non-zero value. The listing:

```
-- subroutine gcd (greatest common divisor)
-- of two non-negative integers by the algorithm of euclid.
-- entry:  a0 = m, a1 = n
-- exit:   a0 = gcd(m, n)
-- working register: t0
fn gcd int (m int, n int) asm {
0:  beq   a1, zero, 1f   -- if n = 0 then return
    remu  t0, a0, a1     -- t0 := remainder of m / n
    addi  a0, a1, 0      -- m := n
    addi  a1, t0, 0      -- n := t0
    jal   zero, 0b       -- repeat the cycle
1:
}
```

The subroutine uses only the base core and the M extension. The register zero (x0) lets several common actions be expressed without separate opcodes: register transfer by addi rd, rs, 0, unconditional transfer by jal zero, gcd (the return address discarded into zero), subroutine return by jalr zero, 0(ra). In practice, the pseudo-instructions mv, j, ret are accepted for these forms and translated by the assembler into the corresponding base instructions.

As an illustration, consider gcd(48, 36). The sequence of states of a0, a1 is given in table 3.

Table 3. Course of the computation of gcd(48, 36)

Step	a0 (m)	a1 (n)	Remainder t0 = m mod n
Initial	48	36	—
1	36	12	12
2	12	0	0
Return	12	—	—

On exit, a0 contains 12, the gcd. The subroutine accesses no memory and uses no saved registers, and may therefore be called from any context with no overhead for state preservation.

substantiation of the choice

Taken together, the properties set out above make RV64IM a rational base for the central processor of the project.

- Regularity and small extent. The small number of instructions and the uniformity of their encoding give a compact, regular control unit and a system that one developer can wholly grasp.
- Predictability of execution. Most instructions execute in a fixed number of clock cycles; there are no complex microprogrammed operations. Timings are predictable, and the analysis and verification of programs are simpler.
- RV64I/M is an open standard, not controlled by any one company. The project is therefore not tied to the policy of a single architecture owner.
- Portability of the software. The unified 64-bit architecture, with direct addressing of large memory volumes, allows programs to be developed in a single high-level language and translated without binding to the features of a particular machine sample.
- Feasibility of emulation. The small size of the base instruction set allows full functional emulation in software by the present collective. This supports both program development before the hardware is available and in-service verification of the equipment.

conclusion

The 64-bit reduced-instruction-set processor RV64IM — by regularity, predictability, openness, and portability — answers the requirements for the base architecture of an open computing complex. The instruction set, small as it is, is functionally complete for the principal classes of computing tasks.

aldos from a bird's eye view

ortfero

abstract

The operating system aldos for the aldan-4 machine is considered. Its composition is set out — the machine, the firmware, the resident almac runtime, aldos itself as a set of almac modules loaded from disk, the core library and the programs, the environment loom among them — together with the sequence of the bringing-up of the machine.

composition of the system

The system is built in layers, lowest first.

Layer	What it is
Machine	A 64-bit RISC-V computer with 16 MB of memory: a screen, a keyboard, a disk, sound, a network and a host directory
Firmware	The boot ROM. It owns every driver and publishes one call table
Almac	The resident binary, and the only one: the language, its compiler and the emergency prompt
Aldos	The task, memory, volume, file and service managers, written in almac

Layer	What it is
Core	The library every program draws on
Programs	The applications, loom among them

The machine has one hart and runs in machine mode only. Its 16 MB of memory begin at `0x80000000`, the first 128 KB of them the boot ROM. The devices stand below that address: a timer, an interrupt controller, a serial line, a real-time clock, a power switch, five virtio transports — the host directory, the network, the keyboard, the sound and the display — and an NVMe disk on a PCI bus. Every device finishes its work inside the store that asks for it; only what comes from outside, a key or a frame, arrives later.

The firmware is the lowest layer of software. Resident in the boot ROM, it brings the machine up and loads the resident binary, and it owns all the machine-dependent drivers besides: the console and the rasteriser that draws it onto the display, the keyboard with its key map, the sound output, the disk, the host directory, the network frames, the clock, the serial line, the timer, the power switch, the page allocator of the operating pool, the bulk byte primitives and the debugger's resumption. It exposes them through one call table of seventy slots, whose source stands in the ROM directly after the eight-byte header and which the firmware copies into RAM at boot. No driver is duplicated above it: every higher layer reaches the hardware by calling through this table. The table is copied into RAM rather than called in the ROM so that a slot can be overwritten by a layer that has something

better to put there. Six slots are so overwritten: the yield and the tick, which the task manager takes, and the four through which a module file is read, which the module reader takes.

The almac runtime is the next layer, and the only resident binary. The firmware loads it from the boot volume to a fixed address and transfers control to it; from that moment it is the principal interface to the machine. It is the environment of the procedural language almac: it compiles every module the system is made of, and when nothing else stands it offers an emergency prompt of its own.

Aldos — the operating-system services proper — is not a resident binary but a set of modules written in almac and loaded from disk by the runtime. It manages memory in two registers — whole pages, and the byte-granular blocks a segregated-fit allocator carves out of them — together with fibers under a cooperative scheduler, the named background services that run upon them, the journal the services write into, the two mounted drives — the boot volume `a` and the host directory `b` — and a small network stack of ARP, IPv4 and UDP over the frames the firmware moves. It touches no device directly: each module that reaches the hardware is a thin adapter over the firmware call table.

The timer is tickless. No interrupt arrives at a fixed rate; the scheduler arms one for the earliest deadline anyone holds, and for nothing when nobody holds one, so an idle machine takes no interrupt until a device has something to say. Every interrupt, a device's or the timer's, ends in the tick slot, and the

tick only services: it wakes the fibers whose device has finished or whose deadline has come, and never switches fibers itself.

The core library is the body of modules a program names by import — among them the clock, the screen, tasks, the heap, files and directories, text and time. It reaches the operating-system services not through any fixed slot but by importing the aldos modules directly, through the almac module system. Programs, the core library and aldos alike are modules of the one almac environment, sharing its address space — almac programs are not processes of the system.

The programs are modules of the same kind, residing under `/a/modules/` on the boot volume, one directory to a sort: `compression`, `demos`, `files`, `games`, `graphics`, `loom`, `math`, `net`, `sound`, `system`, `text`, `time`. The operator's start-up script imports most of them at the start of every session and binds them to bare names, so that the prompt reads as a shell:

```
edit '/a/codex/markoff.moff   -- open the file in an editor
read '/a/codex/markoff.moff   -- read it in a reader
snake                         -- play the game
```

What the script does not import, the operator imports by hand and calls through the module's own name. The two programs listed in this issue are run that way:

```
import 'modules/math/sieve
sieve.bench                   -- report the machine's speed in mips
import 'modules/demos/golife
golife.play                   -- watch the cells
```

The prompt itself belongs to loom, the environment. Its panes stand in a strip, each half the screen wide or the whole of it, and the screen is a window onto the strip: the repl pane, always first, where commands are typed; up to four file listings, four editors and four readers; and one pane each for a picture, a tune, the journal of the services and the debugger.

the boot sequence: from power-on to the prompt

The machine is brought up from power-on to the prompt in seven stages, each holdable in mind. The addresses below are those of the fixed memory map of the aldan-4.

- Power-on. The processor begins execution at `0x80000000`, the start of the boot ROM, where the firmware `bootmgr` resides; a0 holds the hart id and a1 the address of the device tree, which the host writes at `0x80800000` before every reset. The firmware sets the stack pointer to the base of the operating pool at `0x800C0000` — the stack grows downward from there, and the almac runtime later reuses it — and jumps over its eight-byte header (version, edition, date) and the source of its call table.
- The firmware brings the machine to a usable state. It reads the device tree, publishes its call table, and brings up each device in turn. The sub-sequence is given below.
- The firmware loads the resident binary from disk. It walks the root directory of the boot volume, finds

the almac runtime `almac.bin` by name, reads it to `0x80050000`, and transfers execution there. On failure, a diagnostic is written to the screen and the processor halts.

- The almac runtime establishes itself. On receiving control it registers its primitive types and built-in operations, so that the language is ready to compile.
- The runtime reads its start-up script. Reading a file is ordinarily the work of the aldos volume manager and the filesystem beneath it — but those are among the very modules the script is about to bring up, so the first reads cannot yet lean on them. The runtime therefore reads `/a/rc.alm`, and every module the script imports, directly through the firmware call table: the same root-directory primitives the firmware used a moment earlier to load almac.bin, which serve the boot volume alone. The last import of the script is `aldos/modreader`, whose setup points the four module-reading slots at the volume manager; from then on every file operation passes through aldos, and an import may name any mounted drive, the host directory included.
- Aldos comes up as its modules are loaded and first used. The sub-sequence is given below.
- The script defines the operator's commands, and its last line starts loom, which takes the screen and does not return. A fault in the script is reported as a module, a line and a reason, and leaves the machine at the emergency prompt of the runtime, where typing `loom` goes back to the environment. A

machine without a screen stays at that prompt, which the firmware serves on the serial line.

The bringing of the machine to a usable state by the firmware, given as one step above, proceeds in this sequence:

- An early trap handler that reports on the serial line, and the floating-point and vector units switched on;
- The scan of the device tree, which fills the firmware's table of device addresses and interrupt sources;
- Publication of the call table: the seventy routine pointers are copied from the boot ROM into their fixed place in RAM, done before anything can use them;
- The serial line, the shared console cursor, and the page allocator of the operating pool, with all its pages marked free;
- The display, through virtio-gpu: its scale is read, its plane claimed at the head of the pool, and its glyph atlas built;
- The disk: the PCI bus scanned, the NVMe controller enabled, and its queues created; then the block bitmap of the boot volume, sized by the volume's superblock and claimed in the pool next to the plane;
- The host directory, the network, the keyboard and the sound, each over its virtio transport;
- The routing of the device interrupts through the interrupt controller, once their drivers are up;

- The clearing of the screen, and the installation of the trap table.

The coming-up of aldos, given as one step above, is not a single boot-time pass. Each module comes up as it is loaded, in its setup routine, or on its first use:

- The task manager is fitted to the machine as it loads: its setup writes the scheduler into the firmware's yield and tick slots, and its teardown puts the firmware's own routines back before a reset reclaims the code they point at. On the first yield it seeds the boot fiber, which runs on the runtime's own stack;
- The memory manager binds itself to the firmware's page allocator; above it a byte allocator carves page regions into blocks of any size in constant time, so that a claim of a hundred bytes does not cost a whole page. Only a claim past 16 KB goes to the pages directly, where it is a page multiple in practice anyway;
- The volume manager mounts the boot volume as drive `a` and the host directory as drive `b`; disk transfers put the calling fiber to sleep, and the interrupt of the disk wakes it when the transfer is done.

The fixed memory map of the aldan-4, which the operator does well to hold in mind through the whole bringing-up, is given below.

Address	Contents
`0x00100000`	Devices: power switch, clock, timer, interrupt controller, serial line, virtio transports
`0x30000000`	PCI configuration space, then from `0x40000000` the PCI memory window that holds the disk controller
`0x80000000`	Firmware `bootmgr` (boot ROM, 128 KB: header, call-table source, drivers, loader)
`0x80022000`	System buffers (184 KB: firmware state, disk and block-cache buffers, driver queues, sound rings, video state)
`0x8002FC00`	Firmware call table (seventy slots, in RAM, among the system buffers)
`0x80050000`	Almac runtime, and beneath it the stack it and the firmware share (resident region, 448 KB)
`0x800C0000`	Operating pool (15616 KB = 3904 x 4 KB pages); its head holds the screen plane and the disk bitmap, the rest is the heap

conclusion

The article has set out the composition of aldos — the machine, the firmware with its call table of drivers, the resident almac runtime, aldos itself as a set of almac modules loaded from disk, the core library, and the programs with loom among them — and the seven-stage bringing-up of the machine from power-on to the prompt. This arrangement answers the principle of surveyability set out in the opening article.

benchmarking with eratosthenes' sieve

```
--
-- sieve -- prime number stream and cpu benchmark
-- usage:
--      sieve.bench
--      sieve.print 100
-- authors:
--      ortfero <ortfero@gmail.com>, 2026
-- license: cc-by-sa 4.0
--

import 'core/boolean, 'core/heap, 'core/clock,
  'core/task, 'core/tty, 'core/collections/bitset,
  'aldos/machine

type stream record {
  bits bitset.ty, count int, current int
}

fn create (into s stream, upto int)
  raises heap.fault
fn dispose (from s stream)
fn reset (var s stream)
fn next bool (var s stream, var p int)
fn print (upto int)
  raises heap.fault
fn bench int ()
  raises boolean.fault | heap.fault

...


-- set `s` up to stream the primes not exceeding `upto`
fn create (into s stream, upto int)
  raises heap.fault
  final c int {
  -- one flag per odd number 3 <= 2*i + 3 <= upto; the
  -- flags track only odds, so -1 keeps 2 pending
  c = if upto > 2 { (upto - 1) / 2; } else { 0; };
  s = {
```

```
    bits: (bitset.create into, c),
    count: c,
    current: if upto < 2 { 0; } else { -1; }
  };
}


-- dispose the stream. `from` consumes it: bitset.dispose
-- likewise consumes s.bits, and the caller's stream dies
-- at the call.
fn dispose (from s stream) {
  bitset.dispose from s.bits;
}


-- re-arm a drained stream for another full pass:
-- clear every flag and rewind to the pending 2
fn reset (var s stream) {
  bitset.reset var s.bits;
  s.current = -1;
}


-- write the next prime into `p` and return true,
-- or false when the stream is drained
fn next bool (var s stream, var prime int)
  var c int, k int, final n int {

  if s.current < 0 {
    s.current = 0;
    prime = 2;
    return true;
  }

  c = s.current;
  n = s.count;
  for c != n; c += 1 {
    if bitset.test s.bits, c { continue; }

    -- flag c is the prime 2*c + 3; cross out its odd
    -- multiples so later scans skip them. the stride is
    -- `prime` because each step advances 2*prime in
    -- value space, skipping the evens we never tracked.
```

```
    prime = c + c + 3;
    k = c + prime; for k < n; k += prime {
      bitset.set var s.bits, k;
    }

    s.current = c + 1;
    return true;
  }
  s.current = c;
  false;
}


-- print the primes not exceeding `upto`
fn print (upto int)
  raises heap.fault
  var s stream, prime int, more bool {

  create into s, upto;
  defer dispose from s;

  more = next var s, var prime;
  for more; more = next var s, var prime {
    tty.sayin prime, " ";
  }
  tty.say "";
}


const bench_upto   = 16381  -- 8190 odd flags
const primes_count = 1900
const nof_samples  = 20
const window_ms    = 50

-- benchmark the sieve and return the maximum
-- executed millions of instructions per second
fn bench int ()
  raises boolean.fault | heap.fault
  var s stream, si int, executed int, ips int,
    best_ips int, actual_ms int,
  prime int, count int, more bool,
  final started clock.tick {
```

```
  create into s, bench_upto;
  defer dispose from s;

  -- check algorithm is correct
  more = next var s, var prime;
  for more; more = next var s, var prime {
    count += 1;
  }
  if count != primes_count {
    raise boolean.fault.contract_failed;
  }

  -- calculate several times to get best ips
  for si != nof_samples; si += 1 {
    started = clock.now;
    executed = machine.instret;

    -- drain the stream over and over for a full window
    actual_ms = clock.elapsed_ms started;
    for actual_ms <= window_ms;
        actual_ms = clock.elapsed_ms started {
      reset var s;
      more = next var s, var prime;
      for more; more = next var s, var prime { }
    }
    executed = machine.instret - executed;

    -- recalculate to instructions per second
    ips = (executed * 1000) / actual_ms;
    if ips > best_ips { best_ips = ips; }
    task.yield;
  }

  -- report whole mips
  best_ips / 1000000;
}
```

trifles

print current date

jukki

From `/a/rc/commands.alm`:

```
import 'core/bytes, 'core/time, 'core/tty

var scratch [256]byte

fn date ()
  raises bytes.fault {
  tty.say (time.format var scratch, time.now);
}
```

change current directory

jukki

```
import 'core/bytes, 'core/directory, 'core/tty

fn cd (path string)
  raises bytes.fault {
  tty.say (directory.change path);
}
```

create file if not exists

jukki

```
import 'core/file

fn touch (path string)
```

```
  raises file.fault
  final h file.id {
  if file.is_exist path { return none; }
  h = file.create path;
  file.close from h;
}
```

what a function equals

jukki

The first almac function I read looked unfinished — the word that announces the result was nowhere on the page. It was whole all the same, and I wondered how it managed without. So: how a function delivers its value, and why return comes up less often than you might expect.

the value of a function is its last expression

Here is the main thing, and most of the rest follows from it. The body of a function is what sits between its braces, `{` and `}`. The value of the function is the value of the last expression in its body. No keyword, no marker — it is enough that the right expression is evaluated last.

Look at adding two integers:

```
fn add int (a int, b int) { a + b; }
```

There is no `return` here, and still the function returns the sum. `a + b` is the last expression in the body (here it is also the only one), so its value is what goes back to the caller. If this feels strange at first, try reading the body as a claim: "compute `a + b` — that is the answer."

then the type is not arbitrary

If the last expression is the result, its type cannot be just anything: it must match the type you promised in the header. Otherwise the promise is broken, and the translator says so at once.

A typical slip — a function declared to return a byte, ending in a string:

```
fn first_byte byte (s string) {
  s;              -- the result came out string,
}                 -- but byte was promised
```

The program never runs; the mismatch is caught during translation. And this is good: the error is found early, while it is still cheap.

when `return` is needed

While the computation runs straight down, top to bottom, no special word is required. But sometimes you want to leave a function early — say, to throw out a degenerate case without going into the rest. That is what `return` is for: it stops the function immediately and hands back the value given with it.

```
fn list_head_value int (head ^node|none) {
    guard head | return 0;
    head^.value;
}
```

If the reference is empty, `return 0` fires and that is the end of it. If not, control reaches `head^.value` quietly, and it comes back by the rule we already know.

So the two ways of returning live together and divide the work by meaning: you take `return` for an early exit, and you leave the ordinary "bottom" result as the last expression. Writing `return` at the very end, where leaving the expression last would do, is superfluous.

One small thing to remember, since we will want it shortly: `return` must always be followed by an expression. A bare `return;`, with nothing after it, the language does not allow.

functions that return nothing

Very many functions are called not for an answer but for an act — to print something, write somewhere, change some state. Such a function has no result at all, and the way to say so is to write nothing where the result type would stand:

```
fn greet (who string) { ... }
```

There is no type name to put there instead. `none` is a literal and not a type — it stands for the absent value — so it cannot be written into a header; the missing type is the whole of the notation.

For such functions the type rule can be relaxed. Nothing leaves the function anyway, so the type of the last expression no longer matters: it can be of any type, and its value is simply discarded.

```
fn greet (who string)
  var n int {
  n = length who;
```

```
  tty.say "hello, ", who;
}
```

Here the last expression, `say "hello, ", who`, is evaluated for its output on the screen. Whatever its type turns out to be, it cannot leak out — the function returns nothing.

leaving such a function early

Now suppose you want to exit a `none` function early too. The hand reaches for an empty return — but we just agreed that `return` without an expression is not allowed. The literal `none` helps:

```
fn announce (x int) {
  if x <= 0 { return none; }
  tty.say "positive: ", x;
}
```

`return none` reads as "leave, handing nothing back." There is an expression after `return`, so the form is satisfied; `none` is compatible with any function that has no result; and what comes back is exactly "nothing." This is the familiar early exit from a function that delivers nothing.

a question to think over

Set two of our observations side by side and a small gap opens.

On one hand, the last expression in a function with no result may be of any type: it is discarded anyway.

On the other, the early exit we wrote carefully as return none, which is the one form the language names for the purpose. So: what does the translator do if, in such a function, you write return with an expression of plainly the wrong type?

```
fn announce (x int) {
    return x + 1;              -- ?
    tty.say "we never reach here";
}
```

Put another way — does the leniency granted to the last expression reach as far as return? Or must return in a function with no result carry only none?

Work it out for yourself, and then settle it at the prompt — it is one line of typing. One hint: ask what such a return can possibly hand back, whatever you have written after it.

conway's game of life

```
--
-- golife -- conway's game of life
-- usage:
--      golife.play
-- authors:
--      ortfero <ortfero@gmail.com>, 2026
-- license: cc-by-sa 4.0
--

import 'core/base/bytes, 'core/system/clock,
  'core/base/random, 'core/base/boolean, 'core/devices/screen,
  'core/system/event, 'core/system/heap, 'core/base/text,
  'core/devices/composer

fn play() raises boolean.fault | bytes.fault | heap.fault
  | composer.fault

...

-- the board is 60 wide so that it and its frame fit the 65
-- columns of a half window; where the window is wider it is
-- centred (see draw)
const board_y       = 3
const board_h       = 34
const board_w       = 60
const grid_size     = 2040 -- board_w * board_h
const tick_ms       = 100
const live_density  = 30 -- for reset


-- state

type cells [grid_size]bool

type state record {
  entropy random.state,
  current cells, next cells,
  population int, generation int,
  quit bool
```

```
}

fn play () raises boolean.fault | bytes.fault | heap.fault
  | composer.fault ...
fn reset (var stt state) raises boolean.fault ...
fn draw (stt state, pump event.pump, w composer.window)
  raises bytes.fault | boolean.fault ...
fn handle_key bool (k screen.key, var stt state,
  var pump event.pump) raises boolean.fault ...
fn step (var stt state) ...
fn count_neighbors int (column int, i int, current cells) ...


-- entry point: open a window, seed, run until quit. every
-- frame is drawn in answer to the pump's draw: a tick or a
-- key that changed something asks for one, and the window
-- asks for one itself when its area changes
fn play ()
  raises boolean.fault | bytes.fault | heap.fault
  | composer.fault var pump event.pump, k screen.key,
  final stt ^state, e event.kind, w composer.window {

  stt = heap.claim state;
  defer heap.release from stt;

  w = composer.open;
  defer composer.close w;

  random.seed var stt^.entropy, ((clock.now) as int);
  reset var stt^;
  event.paced var pump, tick_ms;
  event.watch var pump, w;

  for not stt^.quit {
    e = event.next var pump, var k;
    when e {
    * event.kind.draw: draw stt^, pump, w;
    * event.kind.key: if handle_key k, var stt^, var pump {
        composer.invalidate w;
      }
    * event.kind.tick: step var stt^;
      composer.invalidate w;
    * else:
```

```
      continue;   -- `next` reports no other kind
    }
  }
}


-- reseed cells with live_density% alive
fn reset (var stt state)
  raises boolean.fault var i int, p int {

  with entropy = stt.entropy;
  with current = stt.current;
  for i != grid_size; i += 1 {
    if (random.range var entropy, 0, 100) < live_density {
      p += 1;
      current[i] = true;
    } else {
      current[i] = false;
    }
  }

  stt.population = p; stt.generation = 0;
  stt.quit = false;
}


-- redraw the frame on the window's surface: border, live
-- cells, then present it, the board centred in the columns
-- the window has. a pause is the pump's held beat, so the
-- caption asks it
fn draw (stt state, pump event.pump, w composer.window)
  raises bytes.fault | boolean.fault var x int, y int,
  caption [128]byte, final i int, j int, paused bool,
  board_x int, at int, sf ^screen.surface {

  at = composer.canvas w;
  if at == 0 { return none; }
  sf = (at as address) as ^screen.surface;
  paused = not (event.is_ticking pump);
  board_x = ((composer.columns w) - board_w) / 2;
  screen.clear var sf^;
  screen.bottom var sf^, " space: pause, r: reset, esc: quit";
  screen.frame var sf^, (board_x - 1), (board_y - 1),
```

```
    (board_w + 2), (board_h + 2);

  with cells = stt.current;
  y = 0; for y != board_h; y += 1 {
    x = 0; for x != board_w; x += 1 {
      j = y * board_w + x;
      if not cells[j] { continue; }
      i = (board_y + y) * screen.width + (board_x + x);
      screen.put var sf^, i, 35;        -- '#'

    }
  }

  screen.print var sf^, (board_x - 1), (board_y - 2),
    (text.format var caption, "gen: ", stt.generation,
    "  pop: ", stt.population, "  ", (if paused { "[paused]";
    } else { "";
    }));
  composer.present w, screen.normal;
}


-- one keystroke: toggle pause, trigger reset, or quit. a
-- pause is the pump's own state, so there is no flag of ours
-- beside it to disagree.
fn handle_key bool (k screen.key, var stt state,
  var pump event.pump) raises boolean.fault {

  when k.code {
  * 'q': if k.chord == screen.ctrl_q {
      stt.quit = true;
      return true;
    }
  * screen.key_esc: stt.quit = true;
    return true;
  * ' ': if (event.is_ticking pump) {
      event.hold var pump;
    } else {
      event.resume var pump;
    }
    return true;
  * 'r': reset var stt;
    return true;
```

```
  }

  false;
}

inline fn next_column int (c int) {
  (c + 1) mod board_w;
}


-- advance one generation: update cells, population,
-- generation
fn step (var stt state)
  var i int, p int, column int, final n int, born bool {

  with current = stt.current;
  with next = stt.next;

  p = 0; column = 0;
  i = 0; for i != grid_size; i += 1 {
    n = count_neighbors column, i, current;
    born = if current[i] {
      or (n == 2), (n == 3);
    } else {
      n == 3;
    };
    next[i] = born;
    if born { p += 1; }
    column = next_column column;
  }
  boolean.copy next, var current;
  stt.population = p; stt.generation += 1;
}


-- count live neighbours of cell `i`
fn count_neighbors int (column int, i int, current cells)
  var n int, final left bool, right bool, up bool, down bool {

  n = 0;
  left = column != 0; right = column != board_w - 1;
  up = i >= board_w; down = i < grid_size - board_w;
  if and up, left, current[i - board_w - 1] { n += 1; }
```

```
  if and up, current[i - board_w] { n += 1; }
  if and up, right, current[i - board_w + 1] { n += 1; }
  if and left, current[i - 1] { n += 1; }
  if and right, current[i + 1] { n += 1; }
  if and down, left, current[i + board_w - 1] { n += 1; }
  if and down, current[i + board_w] { n += 1; }
  if and down, right, current[i + board_w + 1] {
    n += 1;
  }
  n;
}
```

from programming language design to computer construction

niklaus wirth

Reprinted from the ACM Turing Award Lectures

Niklaus Wirth, "From Programming Language Design to Computer Construction," 1984 ACM A.M. Turing Award Lecture. In: ACM Turing Award Lectures, Association for Computing Machinery, New York, NY, USA, 2007. ISBN 978-1-4503-1049-9. https://doi.org/10.1145/1283920.1283941

Copyright (c) 2007 by the Association for Computing Machinery, Inc. (ACM). Permission to make digital or hard copies of portions of this work for personal or classroom use is granted without fee provided that the copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Included here by permission.

Niklaus Wirth of the Swiss Federal Institute of Technology (ETH) was presented the 1984 ACM A. M. Turing Award at the association's annual conference in San Francisco in October in recognition of his outstanding work in developing a sequence of innovative computer languages: Euler, Algol-W, Modula, and Pascal. Pascal, in particular, has become significant pedagogically and has established a foundation for future research in the areas of computer language, systems, and architecture. The hallmarks of a Wirth language are its simplicity, economy of design, and high-quality engineering, which result in a language whose notation appears to be a natural extension of algorithmic thinking rather than an extraneous formalism.

Wirth's ability in language design is complemented by a masterful writing ability. In the April 1971 issue of Communications of the ACM, Wirth published a seminal paper on structured programming ("Program Development by Stepwise Refinement") that recommended top-down structuring of programs (i.e., successively refining program stubs until the program is fully elaborated). The resulting elegant and powerful method of exposition remains interesting reading today even after the furor over structured programming has subsided. Two

later papers, "Toward a Discipline of Real-Time Programming" and "What Can We Do about the Unnecessary Diversity of Notation" (published in CACM in August and November 1974, respectively), speak to Wirth's consistent and dedicated search for an adequate language formalism.

The Turing Award, the association's highest recognition of technical contributions to the computing community, honors Alan M. Turing, the English mathematician who defined the computer prototype Turing machine and helped break German ciphers during World War II.

Wirth received his Ph.D. from the University of California at Berkeley in 1963 and was assistant professor at Stanford University until 1967. He has been professor at the ETH Zurich since 1968; from 1982 until 1984 he was chairman of the division of computer science (Informatik) at ETH. Wirth's recent work includes the design and development of the personal computer Lilith in conjunction with the Modula-2 language. In his lecture, Wirth presents a short history of his major projects, drawing conclusions and highlighting the principles that have guided his work.

From NELIAC (via Algol 60) to Euler and Algol W, to Pascal and Modula-2, and ultimately Lilith, Wirth's search for an appropriate

formalism for systems programming yields intriguing insights and surprising results.

It is a great pleasure to receive the Turing Award, and both gratifying and encouraging to receive appreciation for work done over so many years. I wish to thank ACM for bestowing upon me this prestigious award. It is particularly fitting that I receive it in San Francisco, where my professional career began.

Soon after I received notice of the award, my feeling of joy was tempered somewhat by the awareness of having to deliver the Turing lecture. For someone who is an engineer rather than an orator or preacher, this obligation causes some noticeable anxiety. Foremost among the questions it poses is the following: what do people expect from such a lecture? Some will wish to gain technical insight about one's work, or expect an assessment of its relevance or impact. Others will wish to hear how the ideas behind it emerged. Still others expect a statement from the expert about future trends, events, and products. And some hope for a frank assessment of the present engulfing us, either glorifying the monumental advance of our technology or lamenting its cancerous side effects and exaggerations.

In a period of indecision, I consulted some previous Turing lectures and saw that a condensed report about the history of one's work would be quite acceptable. In order to be not just entertaining, I shall try to summarize what I believe I have learned from the past. This choice, frankly, suits me quite well, because neither do I pretend to know more about the future than most others, nor do I like to be proven wrong

afterwards. Also, the art of preaching about current achievements and misdeeds is not my primary strength. This does not imply that I observe the present computing scene without concern, particularly its tumultuous hassle with commercialism.

Certainly, when I entered the computing field in 1960, it was neither so much in the commercial limelight nor in academic curricula. During my studies at the Swiss Federal Institute of Technology (ETH), the only mention I heard of computers was in an elective course given by Ambros P. Speiser, who later became the president of IFIP. The computer Ermeth developed by him was hardly accessible to ordinary students, and so my initiation to the computing field was delayed until I took a course in numerical analysis at Laval University in Canada. But alas, the ALVAC III E machinery was out of order most of the time, and exercises in programming remained on paper in the form of untested sequences of hexadecimal codes.

My next attempt was somewhat more successful: at Berkeley, I was confronted with Harry Huskey's pet machine, the Bendix G-15 computer. Although the Bendix G-15 provided some feeling of success by producing results, the gist of the programming art appeared to be the clever allocation of instructions on the drum. If you ignored the art, your programs could well run slower by a factor of one hundred. But the educational benefit was clear: you could not afford to ignore the least little detail. There was no way to cover up deficiencies in your design by simply buying more memory. In retrospect, the most attractive feature was that every detail of the machine was visible and could

be understood. Nothing was hidden in complex circuitry, silicon, or a magic operating system.

On the other hand, it was obvious that computers of the future had to be more effectively programmable. I therefore gave up the idea of studying how to design hardware in favor of studying how to use it more elegantly. It was my luck to join a research group that was engaged in the development – or perhaps rather improvement – of a compiler and its use on an IBM 704. The language was called NELIAC, a dialect of Algol 58. The benefits of such a "language" were quickly obvious, and the task of automatically translating programs into machine code posed challenging problems. This is precisely what one is looking for when engaged in the pursuit of a doctorate. The compiler, itself written in NELIAC, was a most intricate mess. The subject seemed to consist of 1 percent science and 99 percent sorcery, and this tilt had to be changed. Evidently, programs should be designed according to the same principles as electronic circuits, that is, clearly subdivided into parts with only a few wires going across the boundaries. Only by understanding one part at a time would there be hope of finally understanding the whole.

This attempt received a vigorous starting impulse from the appearance of the report on Algol 60. Algol 60 was the first language defined with clarity; its syntax was even specified in a rigorous formalism. The lesson was that a clear specification is a necessary but not sufficient condition for a reliable and effective implementation. Contact with Aadrian van Wijngaarden, one of Algol's codesigners, brought out the central

theme more distinctly: could Algol's principles be condensed and crystallized even further?

Thus began my adventures in programming languages. The first experiment led to a dissertation and the language Euler — a trip with the bush knife through the jungle of language features and facilities. The result was academic elegance, but not much of practical utility — almost an antithesis of the later data-typed and structured programming languages. But it did create a basis for the systematic design of compilers that, so was the hope, could be extended without loss of clarity to accommodate further facilities.

Euler caught the attention of the IFIP working group that was engaged in planning the future of Algol. The language Algol 60, designed by and for numerical mathematicians, had a systematic structure and a concise definition that were appreciated by mathematically trained people but lacked compilers and support by industry. To gain acceptance, its range of application had to be widened. The working group assumed the task of proposing a successor and soon split into two camps. On one side were the ambitious who wanted to erect another milestone in language design, and, on the other, those who felt that time was pressing and that an adequately extended Algol 60 would be a productive endeavor. I belonged to this second party and submitted a proposal that lost the election. Thereafter, the proposal was improved with contributions from Tony Hoare (a member of the same group) and implemented on Stanford University's first IBM 360. The language later became known as Algol W and was used in several universities for teaching purposes.

A small interlude in this sizable implementation effort is worth mentioning. The new IBM 360 offered only assembler code and, of course, Fortran. Neither particularly were loved, either by me or my graduate students, as a tool for designing a compiler. Hence, I mustered the courage to define yet another language in which the Algol compiler could be described: a compromise between Algol and the facilities offered by the assembler, it would be a machine language with Algol-like statement structures and declarations. Notably, the language was defined in a couple of weeks; I wrote the cross compiler on the Burroughs B-5000 computer within four months, and a diligent student transported it to the IBM 360 within an equal period of time. This preparative interlude helped speed up the Algol effort considerably. Although envisaged as serving our own immediate needs and to be discarded thereafter, it quickly acquired its own momentum. PL360 became an effective tool in many places and inspired similar developments for other machines.

Ironically, the success of PL360 was also an indication of Algol-W's failure. Algol's range of application had been widened, but as a tool for systems programming, it still had evident deficiencies. The difficulty of resolving many demands with a single language had emerged, and the goal itself became questionable. PL/1, released around this time, provided further evidence to support this contention. The Swiss army knife idea has its merits, but if driven to excess, the knife becomes a millstone. Also, the size of the Algol-W compiler grew beyond the limits within which one could rest comfortably with the feeling of

having a grasp, a mental understanding, of the whole program. The desire for a more concise yet more appropriate formalism for systems programming had not been fulfilled. Systems programming requires an efficient compiler generating efficient code that operates without a fixed, hidden, and large so-called run-time package. This goal had been missed by both Algol-W and PL/1, both because the languages were complex and the target computers inadequate.

In the fall of 1967, I returned to Switzerland. A year later, I was able to establish a team with three assistants to implement the language that later became known as Pascal. Freed from the constraints of obtaining a committee consensus, I was able to concentrate on including the features I myself deemed essential and excluding those whose implementation effort I judged to be incommensurate with the ultimate benefit. The constraint of severely limited manpower is sometimes an advantage.

Occasionally, it has been claimed that Pascal was designed as a language for teaching. Although this is correct, its use in teaching was not the only goal. In fact, I do not believe in using tools and formalisms in teaching that are inadequate for any practical task. By today's standards, Pascal has obvious deficiencies for programming large systems, but 15 years ago it represented a sensible compromise between what was desirable and what was effective. At ETH, we introduced Pascal in programming classes in 1972, in fact against considerable opposition. It turned out to be a success because it allowed the teacher to concentrate more heavily on structures and concepts than features and

peculiarities, that is, on principles rather than techniques.

Our first Pascal compiler was implemented for the CDC 6000 computer family. It was written in Pascal itself. No PL6000 was necessary, and I considered this a substantial step forward. Nonetheless, the code generated was definitely inferior to that generated by Fortran compilers for corresponding programs. Speed is an essential and easily measurable criterion, and we believed the validity of the high-level language concept would be accepted in industry only if the performance penalty were to vanish or at least diminish. With this in mind, a second effort — essentially a one-man effort — was launched to produce a high-quality compiler. The goal was achieved in 1974 by Urs Ammann, and the compiler was thereafter widely distributed and is being used today in many universities and industries. Yet the price was high; the effort to generate good (i.e., not even optimal) code is proportional to the mismatch between language and machine, and the CDC 6000 had certainly not been designed with high-level languages in mind.

Ironically again, the principal benefit turned up where we had least expected it. After the existence of Pascal became known, several people asked us for assistance in implementing Pascal on various other machines, emphasizing that they intended to use it for teaching and that speed was not of overwhelming importance. Thereupon, we decided to provide a compiler version that would generate code for a machine of our own design. This code later became known as P-code. The P-code version was very easy to construct because the

new compiler was developed as a substantial exercise in structured programming by stepwise refinement and therefore the first few refinement steps could be adopted unchanged. Pascal-P proved enormously successful in spreading the language among many users. Had we possessed the wisdom to foresee the dimensions of this movement, we would have put more effort and care into designing and documenting P-code. As it was, it remained a side effort to honor the requests in one concentrated stride. This shows that even with the best intentions one may choose one's goals wrongly.

But Pascal gained truly widespread recognition only after Ken Bowles in San Diego recognized that the P-system could well be implemented on the novel microcomputers. His efforts to develop a suitable environment with integrated compiler, filer, editor, and debugger caused a breakthrough: Pascal became available to thousands of new computer users 'who were not burdened with acquired habits or stifled by the urge to stay compatible with software of the past.

In the meantime, I terminated work on Pascal and decided to investigate the enticing new subject of multiprogramming, where Hoare had laid respectable foundations and Brinch Hansen had led the way with his Concurrent Pascal. The attempt to distill concrete rules for a multiprogramming discipline quickly led me to formulate them in terms of a small set of programming facilities. In order to put the rules to a genuine test, I embedded them in a fragmentary language, whose name was coined after my principal aim: modularity in program systems. The module later turned out to be the principal asset of this language; it gave

the abstract concept of information hiding a concrete form and incorporated a method as significant in uniprogramming as in multiprogramming. Also, Modula contained facilities to express concurrent processes and their synchronization.

By 1976, I had become somewhat weary of programming languages and the frustrating task of constructing good compilers for existing computers that were designed for old-fashioned "by-hand" coding. Fortunately, I was given the opportunity to spend a sabbatical year at the research laboratory of Xerox Corporation in Palo Alto, where the concept of the powerful personal workstation had not only originated but was also put into practice. Instead of sharing a large, monolithic computer with many others and fighting for a share via a wire with a 3-kHz bandwidth, I now used my own computer placed under my desk over a 15-MHz channel. The influence of a 5000-fold increase in anything is not foreseeable; it is overwhelming. The most elating sensation was that after 16 years of working for computers, the computer now seemed to work for me. For the first time, I did my daily correspondence and report writing with the aid of a computer, instead of planning new languages, compilers, and programs for others to use. The other revelation was that a compiler for the language Mesa whose complexity was far beyond that of Pascal, could be implemented on such a workstation. These new working conditions were so many orders of magnitude above what I had experienced at home that I decided to try to establish such an environment there as well.

I finally decided to dig into hardware design. This decision was reinforced by my old disgust with existing

computer architectures that made life miserable for a compiler designer with a bent toward systematic simplicity. The idea of designing and building an entire computer system consisting of hardware, microcode, compiler, operating system, and program utilities quickly took shape in my imagination — a design that would be free from any constraint to be compatible with a PDP-11 or an IBM 360, or Fortran, Pascal, Unix, or whatever other current fad or committee standard there might be.

But a sensation of liberation is not enough to succeed in a technical project. Hard work, determination, a sensitive feeling of what is essential and what ephemeral, and a portion of luck are indispensable. The first lucky accident was a telephone call from a hardware designer enquiring about the possibility of coming to our university to learn about software techniques and acquire a Ph.D. Why not teach him about software and let him teach us about hardware? It didn't take long before the two of us became a functioning team, and Richard Ohran soon became so excited about the new design that he almost totally forgot both software and Ph.D. That didn't disturb me too much, for I was amply occupied with the design of hardware parts; with specifying the micro- and macrocodes, and by programming the latter's interpreter; with planning the overall software system; and in particular with programming a text editor and a diagram editor, both making use of the new high-resolution bit-mapped display and the small miracle called mouse as a pointing device. This exercise in programming highly interactive utility programs

required the study and application of techniques quite foreign to conventional compiler and operating system design.

The total project was so diversified and complex that it seemed irresponsible to start it, particularly in view of the small number of part-time assistants available to us, who averaged around seven. The major threat was that it would take too long to keep the enthusiastic two of us persisting and to let the others, who had not yet experienced the power of the workstation idea, become equally enthusiastic. To keep the project within reasonable dimensions, I stuck to three dogmas: aim for a single-processor computer to be operated by a single user and programmed in a single language. Notably, these cornerstones were diametrically opposed to the trends of the time, which favored research in multiprocessor configurations, time-sharing multiuser operating systems, and as many languages as you could muster.

Under the constraints of a single language, I faced a difficult choice whose effects would be wide ranging, namely, that of selecting a language. Of existing languages, none seemed attractive. Neither could they satisfy all the requirements, nor were they particularly appealing to the compiler designer who knows the task has to be accomplished in a reasonable time span. In particular, the language had to accommodate all our wishes with regard to structuring facilities, based on 10 years' experience with Pascal, and it had to cater to problems so far only handled by coding with an assembler. To cut a long story short, the choice was to design an offspring of both proven

Pascal and experimental Modula, that is, Modula-2. The module is the key to bringing under one hat the contradictory requirements of high-level abstraction for security through redundancy checking and low-level facilities that allow access to individual features of a particular computer. It lets the programmer encapsulate the use of low-level facilities in a few small parts of the system, thus protecting him from falling into their traps in unexpected places.

The Lilith project proved that it is not only possible but advantageous to design a single-language system. Everything from device drivers to text and graphics editors is written in the same language. There is no distinction between modules belonging to the operating system and those belonging to the user's program. In fact, that distinction almost vanishes and with it the burden of a monolithic, bulky resident block of code, which no one wants but everyone has to accept. Moreover, the Lilith project proved the benefits of a well-matched hardware/software design. These benefits can be measured in terms of speed: comparisons of execution times of Modula programs revealed that Lilith is often superior to a VAX 750 whose complexity and cost are a multiple of those of Lilith. They can also be measured in terms of space: the code of Modula programs for Lilith is shorter than the code for PDP-11, VAX, or 68000 by factors of 2 to 3, and shorter than that of the NS 32000 by a factor of 1.5 to 2. In addition, the code-generating parts of compilers for these microprocessors are considerably more intricate than they are in Lilith due to their ill-matched instruction sets. This length factor has to

be multiplied by the inferior density factor, which casts a dark shadow over the much advertised high-level language suitability of modern microprocessors and reveals these claims to be exaggerated. The prospect that these designs will be reproduced millions of times is rather depressing, for by their mere number they become our standard building blocks. Unfortunately, advances in semiconductor technology have been so rapid that architectural advances are overshadowed and have become seemingly less relevant. Competition forces manufacturers to freeze new designs into silicon long before they have proved their effectiveness. And whereas bulky software can at least be modified and at best be replaced, nowadays complexity has descended into the very chips. And there is little hope that we have a better mastery of complexity when we apply it to hardware rather than software.

On both sides of this fence, complexity has and will maintain a strong fascination for many people. It is true that we live in a complex world and strive to solve inherently complex problems, which often do require complex mechanisms. However, this should not diminish our desire for elegant solutions, which convince by their clarity and effectiveness. Simple, elegant solutions are more effective, but they are harder to find than complex ones, and they require more time, which we too often believe to be unaffordable.

Before closing, let me try to distill some of the common characteristics of the projects that were mentioned. A very important technique that is seldom used as effectively as in computing is the bootstrap. We used it in virtually every project. When developing

a tool, be it a programming language, a compiler, or a computer, I designed it in such a way that it was beneficial in the very next step: PL360 was developed to implement Algol W; Pascal to implement Pascal; Modula-2 to implement the whole workstation software; and Lilith to provide a suitable environment for all our future work, ranging from programming to circuit documentation and development, from report preparation to font design. Bootstrapping is the most effective way of profiting from one's own efforts as well as suffering from one's mistakes. This makes it mandatory to distinguish early between what is essential and what ephemeral. I have always tried to identify and focus in on what is essential and yields unquestionable benefits. For example, the inclusion of a coherent and consistent scheme of data type declarations in a programming language I consider essential, whereas the details of varieties of for-statements, or whether the compiler distinguishes between upper- and lowercase letters, are ephemeral questions. In computer design, I consider the choice of addressing modes and the provision of complete and consistent sets of {signed and unsigned} arithmetic instructions including proper traps on overflow to be crucial; in contrast, the details of a multichannel prioritized interrupt mechanism are rather peripheral. Even more important is ensuring that the ephemeral never impinge on the systematic, structured design of the central facilities. Rather, the ephemeral must be added fittingly to the existing, well-structured framework.

Rejecting pressures to include all kinds of facilities that "might also be nice to have" is

sometimes hard. The danger that one's desire to please will interfere with the goal of consistent design is very real. I have always tried to weigh the gains against the cost. For example, when considering the inclusion of either a language feature or the compiler's special treatment of a reasonably frequent construct, one must weigh the benefits against the added cost of its implementation and its mere presence, which results in a larger system. Language designers often fail in this respect. I gladly admit that certain features of Ada that have no counterparts in Modula-2 may be nice to have occasionally, but at the same time, I question whether they are worth the price. The price is considerable: first, although the design of both languages started in 1977, Ada compilers have only now begun to emerge, whereas we have been using Modula since 1979. Second, Ada compilers are rumored to be gigantic programs consisting of several hundred thousand lines of code, whereas our newest Modula compiler measures some five thousand lines only. I confess secretly that this Modula compiler is already at the limits of comprehensible complexity, and I would feel utterly incapable of constructing a good compiler for Ada. But even if the effort of building unnecessarily large systems and the cost of memory to contain their code could be ignored, the real cost is hidden in the unseen efforts of the innumerable programmers trying desperately to understand them and use them effectively.

Another common characteristic of the projects sketched was the choice of tools. It is my belief that a tool should be commensurate with the product; it must

be as simple as possible, but no simpler. A tool is in fact counterproductive when a large part of the entire project is taken up by mastering the tool. Within the Euler, Algol W, and PL360 projects, much consideration was given to the development of table-driven, bottom-up syntax analysis techniques. Later, I switched back to the simple recursive-descent, top-down method, which is easily comprehensible and unquestionably sufficiently powerful, if the syntax of the language is wisely chosen. On the development of the Lilith hardware, we restricted ourselves to a good oscilloscope; only rarely was a logic state analyzer needed. This was possible due to a relatively systematic, trick-free concept for the processor.

Every single project was primarily a learning experiment. One learns best when inventing. Only by actually doing a development project can I gain enough familiarity with the intrinsic difficulties and enough confidence that the inherent details can be mastered. I never could separate the design of a language from its implementation, for a rigid definition without the feedback from the construction of its compiler would seem to me presumptuous and unprofessional. Thus, I participated in the construction of compilers, circuitry, and text and graphics editors, and this entailed microprogramming, much high-level programming, circuit design, board layout, and even wire wrapping. This may seem odd, but I simply like hands-on experience much better than team management. I have also learned that researchers accept leadership from a factual, in-touch team member much more readily than from an organization expert, be he a manager in

industry or a university professor. I try to keep in mind that teaching by setting a good example is often the most effective method and sometimes the only one available.

Lastly, each of these projects was carried through by the enthusiasm and the desire to succeed in the knowledge that the endeavor was worthwhile. This is perhaps the most essential but also the most elusive and subtle prerequisite. I was lucky to have team members who let themselves be infected with enthusiasm, and here is my chance to thank all of them for their valuable contributions. My sincere thanks go to all who participated, be it in the direct form of working in a team, or in the indirect forms of testing our results and providing feedback, of contributing ideas through criticism or encouragement, or of forming user societies. Without them, neither Algol W, nor Pascal, nor Modula-2, nor Lilith would have become what they are. This Turing Award also honors their contributions.

on process isolation

to the editor

progzmaker

I held off writing for some time, because the observations I wish to share have long been considered platitudes in our trade, and platitudes are not interesting to repeat. But reading the recent polemic on the design of operating systems, I noticed that these platitudes provoke doubts in some of your readers, and I decided after all to set them out as I understand them, from my own vantage point. My vantage point is fifteen years in a large computing center, where systems run not for elegance but because without them quite a lot else does not run either.

The subject is the separation of address spaces between processes — that same isolation that mature systems install without discussion. I want to say, positively, why it is installed, without arguing with anyone in particular.

The first and simplest point. Programs make mistakes. Not in theory but every day, and not from the author's irresponsibility, but because a person who has written three thousand lines cannot hold all their interactions in his head. A wild pointer in one program, left without obstruction, corrupts the memory of another program or of the kernel, and the result is a system in an undefined state. Isolation turns this into "this particular program has crashed," and the

culprit points at itself. From here begins the possibility of debugging, and the possibility of continuing to work without rebooting everything. This is not even about security; this is about operation.

Second — about security in the simplest sense. On any working machine there is data that one program is trusted with and another is not. The accounting program need not read the correspondence; the correspondence program need not poke into signing keys; the display program need not know passwords. Without a barrier all of this is one common muddle in which every process is a potential observer of everything. Isolation here is not censorship but elementary decency: to each their own, and no one looks into someone else's affairs without invitation.

Third, a point rarely spoken of because there is no drama in it. Process isolation enormously simplifies programming. Every program is written as if it were alone in the machine and owned all memory from zero to infinity. It need not know where others live, need not negotiate placement, need not watch that it does not step on someone else's ground. This is a great unburdening of the author. Secondarily, the system gains the freedom to move programs around in physical memory, swap them to disk, share common pieces between them — and the program need know nothing of it. A convenience for which we long ago stopped being grateful, because we got used to it.

Fourth — about the data a program receives from outside. Any nontrivial program reads something: a file, a message, a picture, a stream. And at the moment of reading it is forced to trust the contents precisely

insofar as its parser is well written. Parsers are written by people; people make mistakes; mistakes in parsers turn foreign data into the execution of foreign intent inside the program. Without isolation, this foreign intent executes with the rights of the whole system. With isolation, with the rights of a single process which by design has nothing extra. This is the difference between "we have lost a page" and "we have lost everything."

Fifth. A barrier is not a guarantee, and no one who works with it claims otherwise. A barrier can be breached, and sometimes is. But exploitation is an economic occupation: the attacker spends time, talent and money on each layer that gets in his way. A barrier raises the cost of attack by factors, sometimes by orders of magnitude, and cuts off almost everyone who has nothing to pay with. A defense that cuts off ninety-nine percent of attackers and raises the remaining one percent's work to unaffordable sums is no trifle, even if it is not absolute. Absolute defenses do not exist in our craft at all.

Sixth, a finer point. Between the moment a bug enters a program and the moment it becomes known, time passes — on serious systems we are accustomed to count it in years, and there is no exaggeration here. Through those years the bug sits quietly in working code. If there is no defense, then anyone who has found that bug before the rest of the community owns everything those systems do for the entire length of the window. Isolation is insurance for exactly that length. Not against "it will never happen," but against "when it happens, let it not catch us all at once."

Seventh. Isolation is useful not only between different programs but inside a single program, if it is large enough. The part that parses incoming data should be separated from the part that stores the result; the part that talks to the network from the part that talks to the disk. Every such internal barrier limits what an attacker obtains if he manages to subvert the logic of one component. On mature projects this has long been simply good form — to separate privileges within one's own application without waiting for the kernel to separate them for you.

Eighth — an observation, not a theory. The most independent practitioners, those who build their own systems end to end, with no eye to the industry, for the sake of their own freedom and their own understanding, arrive at isolation in exactly the same way as we do. Not because they have been told to, but because, having lived with their own machine long enough, they begin to see in it something of their own rather than something foreign. This is not an argument from authority; it is an argument from convergence: different people with different motives, starting from different places, arrive at the same thought, and that means something.

And last, the most personal. I install barriers not because I distrust my own code, but because I trust it exactly as much as I trust myself. And I have known myself for twenty-some years and can say with confidence: I make mistakes, and I will make mistakes, and on a Friday evening I will make them especially. A barrier is the instrument by which I reduce the

consequences of my own inevitable fallibility in advance, while my head is still clear and I can still think about it. This is not self-deprecation, and not insurance against the vendor. It is professional maturity: the longer you are in the craft, the more calmly you admit that the safety harness is needed, and the more willingly you put it on.

There is no new knowledge in this letter. Everything I have set out has long been known to anyone who has lived through a few production cycles and seen how simplifications dictated by aesthetics end. But platitudes are sometimes worth saying aloud, especially in a journal that is read by the young. If even one of them, having read this letter, pauses before tearing down the isolation in his next project, then consider that my shift today was not in vain.

to the progzmaker

ortfero

We read progzmaker's letter with close attention, and we wish to reply — because, as a small research project without significant resources, we are granted an unusual privilege in this profession: to re-examine settled claims and norms.

Let us state plainly: we hold none of progzmaker's arguments for isolation to be false. Programs do err. Hostile input is dangerous. A barrier does raise the cost of attack. The window between the appearance of a defect and its discovery does indeed run to years. The personal maturity that prompts one to fasten one's own

safety harness is indeed an admirable thing. To dispute these points is pointless, and we shall not.

We wish only to note that isolation, like any engineering decision, has another side, at which long custom has trained us not to look. Not in order to abolish it, but so that, at least once, we may see it as a choice rather than a law of nature.

First — the historical point. The separation of address spaces between processes was born not from concern for the user, but from the necessity of dividing an expensive machine among mutually-distrusting parties. It was the era of one computer to a hundred people, in which one part of the machine's population was, by office, not to see what another part was doing. Upon this primary meaning later meanings were accreted: protection from one's own errors, fault localisation, security against hostile input. All are legitimate. But the primary meaning — the separation of people — is simply unnecessary on the personal machine of a single owner; there is no one to distrust. That we continue, by inertia, to build machines as though for many, is an inheritance worth noticing at least once.

Second — on where the barrier runs. On a contemporary personal machine the most valuable thing is not the system files but the home directory: keys, correspondence, documents, sessions. The classical UID-based separation does not protect this, because all of a user's processes run under the same UID and read one another through the filesystem. That is, the isolation for whose sake the model was built is, in our reality, drawn not where the attacked thing actually lies. This is no reproach to isolation as an idea; it is an

observation: the boundary is drawn not along the line on which the real risk runs.

Third — on alternative implementation. That barriers between programs are useful is not in question for us; we ask where to set them, and by what means. Hardware separation of pages is one implementation, and it has a price: in silicon, in context switches, in kernel complexity, in the mitigations one must layer on after each new finding in speculative execution. Language safety is another implementation, and it sets the barriers not in the hardware but in the types: the modules of a program cannot write into one another's memory not because the MMU forbids it, but because the compiler will not let them. This is not the abolition of the idea of isolation; it is its transfer to a floor on which it is cheaper and more exact. In our system the work of isolation is done — only not by the mechanisms to which the large computing centre has grown accustomed.

Fourth — on an inconvenient property of hardware separation, on which we would wish our respected colleague to think as well. It is a structurally asymmetric mechanism. The processor's privileged mode — the very thing that affords isolation — is, by construction, designed so that one party may dictate to another what is permitted. On the operator's machine, where the key to privileged mode is in the owner's hands, it is an instrument of his sovereignty. But by the same mechanism, once the key has moved into the vendor's hands, the owner is forbidden to install and run on his own machine what he wishes. One and the same transistor serves two opposing policies. The machine is

defended from user code — and the vendor is defended from the owner — by one and the same primitive. It seems to us this is worth thinking of when one debates whether the primitive is necessary.

Fifth — on complexity, and on why we have received it. Our respected colleague works in a system whose count runs in millions of lines, and in such a system isolation does indeed become the last reasonable insurance, because to read all of it and hold it in one's head is physically impossible. But this complexity we have received not from the laws of nature. It was produced by the combination of industrial development, staff turnover, "efficiency" metrics, and standards layered over decades under the pressure of various interests pulling in their own directions. A small system, in which the author lives with his code for years and treats it as his own work, is a different regime, and the quality of code within it is qualitatively different. This does not render our colleague's arguments false for his context, but it does mean that part of the weight of his arguments rests upon conditions which, in our context, simply are not reproduced. The small system does not require all the safeguards of the large, precisely because it is small.

Sixth — on new code which has not yet been tried in service. Our respected colleague is right that such code cannot be allowed everything at once. But in our system there is a natural mechanism for this: the emulator upon which the system itself is built. An untried program is launched in an external instance of the virtual machine with a curtailed virtual world —

without access to valuable data, with a fake network, with its own virtual disk. When we have watched it long enough, we resettle it into the main machine, or leave it to live forever in its sandbox. This is not a renunciation of isolation; it is its transfer from the boundary of the process to the boundary of the virtual computer. The boundary turns out to be larger, and it can be drawn precisely along the line on which actual trust runs, rather than along the one drawn for us in the seventies.

And a last thing, of which we wish to speak honestly. We do not claim that our model will defeat our respected colleague's model. We are not, in fact, certain that it will defeat anything beyond our own curiosity. But in an industry in which small research groups do not permit themselves to re-examine foundations, the large computing centres receive nothing upon which, in the next generation, they might stand. Do not, please, deprive us of the right to this question; and if it does not arise for you, consider that we ask it on your behalf.

With respect, and with thanks for raising the conversation.

feedback

"Questions of Cybernetics" is the bulletin of a small working group, and it lives on what comes back. If something in this issue is wrong, unclear, or worth arguing with, we want to hear it. If you have tried the system and found where it breaks, where it surprises you, or where it fails to be as surveyable as we claim, write to us. Corrections are welcome and will be printed. So are disagreements, especially the well-reasoned kind.

We are also interested in programs, listings, notes from your own work, accounts of teaching the machine to someone, and short pieces of fiction or reflection for the reading room. If you have built something on the aldan stack, or ported something to it, or written something about it, send it along. We would rather print a rough piece by a working hand than a polished one by a distant observer.

And if any of this resonates more deeply —- if you find yourself wanting not only to write in but to take part in the work —- the aldan collective is open to new members. The condition is simple: an interest in building small, comprehensible systems. Write to us about yourself and what you would like to do.

aldanteam@protonmail.com. We read everything that arrives.

the guy from hell. chapter 1

arkady strugatsky, boris strugatsky

What a village! I've never seen a village like this and didn't even know such villages existed. The houses are round, brown, windowless, standing on stilts like watchtowers, and underneath them, there's all sorts of junk piled up — huge pots, troughs, rusty cauldrons, wooden rakes, shovels... the ground between the houses is clay, so burnt and trampled that it even shines. And everywhere you look, there are nets. Dry nets. I don't know what they catch with these nets here: swamp on the right, swamp on the left, smells like a dump... a dreadful hole. They've been rotting here for a thousand years and, if not for the duke, would have rotted for another thousand. The north. Wilderness. And, of course, no residents in sight. Either they ran away, were taken, or are hiding.

In the square near the trading post, a field kitchen, taken off its wheels, was smoking. A huge porcupine — as wide as he was tall — in a dirty white apron over his dirty gray uniform, was stirring a pot with a long-handled ladle. I think it was mainly this pot that was causing the village to stink.

We approached, and Cheetah, stopping, asked where the commander was. This creature didn't even turn around — he muttered something into his stew and pointed with the ladle somewhere down the street. I gave him a kick in the backside with the toe of my

boot, and he quickly turned around, saw our uniforms, and immediately stood at attention. His face matched its ham-like body, and he hadn't been shaved for a whole week, this porcupine.

"So where's your commander?" Cheetah asked again, pressing his cane into the porcupine's fat neck under his double chin.

The porcupine rolled his eyes, smacked his lips, and rasped:

"Sorry, sir senior instructor... the staff major is at the positions... please, down this street... right at the outskirts... accept my apologies, sir senior instructor..."

He kept rasping and gurgling something, and from around the corner of the trading post, two more porcupines emerged — even scarier than this one, complete scarecrows without weapons or headgear. They saw us and froze at attention. Cheetah just looked at them, sighed, and then walked on, tapping his cane against his boot.

Yes, we got here just in time. These porcupines, they would have made a mess of things! I've only seen three of them so far, but I'm already sick of them, and it's clear to me that this, excuse the expression, unit, cobbled together from rear-line pests, hastily and haphazardly, with all these regimental bakers, brigade cobblers, clerks, quartermasters, idiots, lazybones, half-blind fools, and funeral detail eagles — they're all walking fertilizer, lubricant for a bayonet. The imperial armored vehicles would have rolled through them without even noticing anyone was there. Easily.

Someone called out to us. To the left, between two houses, there was a camouflage tent set up and a white-green cloth hanging on a pole. A medical post. Two more porcupines were slowly rummaging through green packs of medical supplies, and on mats thrown directly on the ground lay the wounded. There were three wounded in total; one, with his head bandaged, was propped up on his elbow, looking at us. When we turned around, he called out again:

"Sir, instructor! Just a moment, please!..."

We approached. Cheetah squatted down, and I remained standing behind him. The wounded man had no visible insignia; he was wearing a torn, burnt camouflage jumpsuit, unbuttoned on his bare hairy chest. But from his face, from his wild eyes with singed eyelashes, I immediately knew that this one wasn't a porcupine, no, this one was the real deal. And I was right.

"Brigadier-jaeger Baron Tragg," he introduced himself. It sounded like tank treads clanking. "Commander of the independent 18th forest jaeger unit."

"Senior instructor Digga," Cheetah said. "I'm listening, brave brother."

"A cigarette..." the baron asked in a suddenly hoarse voice.

While Cheetah was taking out his cigarette case, he hurriedly continued:

"Got hit by a flamethrower, scorched like a pig... thank god, the swamp was nearby, I submerged up to my eyebrows... but my cigarettes — turned to mush... thanks..."

He took a drag, closing his eyes, and immediately started coughing harshly, turning blue, convulsing. A

drop of blood oozed from under the bandage on his cheek and congealed, like resin. Without turning, Cheetah extended his hand over his shoulder and snapped his fingers. I pulled the flask from my belt and handed it to him. The baron took a few gulps, and he seemed to feel a bit better. The other two wounded lay motionless — either they were asleep or already gone. The medics glanced at us fearfully. Not really looking, just sneaking glances.

"Good..." Baron Tragg said, returning the flask. "How many men do you have?"

"Four tens," Cheetah replied. "Keep the flask... keep it for yourself."

"Forty... forty fighting cats..."

"Kittens," Cheetah said. "Unfortunately... but we'll do everything we can."

The baron looked at him from under his burnt eyebrows. There was agony in his eyes.

"Listen, brave brother," he said. "I have no one left. I've been retreating from the pass for three days. Constant battles. The rat-eaters are coming on armored vehicles. I've burned about twenty of them. The last two — yesterday... right here, on the outskirts... you'll see. This staff major... an idiot and a coward... an old wreck... I wanted to shoot him, but I had no bullets left. Can you imagine? Not a single bullet! He was hiding in the village with his porcupines, watching as we were burned out one by one... what was I saying? Yes! Where is Gagrid's brigade? The radio is shattered... the last message was: "Hold on, Gagrid's brigade is on the way..."

listen, a cigarette... and inform headquarters that the 18th independent is no more."

The baron was already delirious. His wild eyes were clouded, tongue barely moved. He fell onto his back, still talking, talking, mumbling, rasping while his gnarled fingers restlessly groped around, clutching at either the edges of the mat or his jumpsuit. Then he suddenly fell silent mid-sentence, and Cheetah stood up.

He slowly pulled out a cigarette, not taking his eyes off the upturned face, flicked the lighter, then leaned down and placed the cigarette case along with the lighter next to the blackened fingers, which greedily clutched the cigarette case and gripped it tightly. Without a word, Cheetah turned, and we moved on.

I thought that it was probably merciful — the brigadier had lost consciousness just in time. Otherwise, he would have had to hear that Gagrid's brigade was no more either. They were hit overnight by a carpet bombing on the supply route — we spent two hours clearing the highway of wreckage and the already cooling meat of bodies, fending off the mad ones trying to crawl under the trucks to hide. All we found of Gagrid was his general's cap, crusted with blood... a chill ran through me when I remembered all this, and I involuntarily glanced at the sky and felt relieved at how low, gray, and bleak it was.

The first thing we saw as we stepped beyond the outskirts was an imperial armored vehicle that had veered off the road and crashed nose-first into the village well. It had already cooled down, and the grass

around it was coated in thick soot. A dead rat-eater lay face down under the open side hatch — everything on him was burned, except for his ginger boots with triple soles. The rat-eaters have good boots! Their boots are good, their armored vehicles, and maybe their bombers too. But as soldiers, everyone knows they are useless. Jackals.

"How do you like this position, Gug?" asked Cheetah.

I looked around. What a position! I couldn't believe my eyes. The porcupines had dug themselves trenches on both sides of the road, right in the middle of the clearing between the village outskirts and the jungle. The jungle stood like a wall in front of the trenches, about fifty paces away, no more. You could amass a regiment, a brigade, whatever you wanted there, and the trenches wouldn't know a thing about it. And when they did find out, it would be too late to do anything. The trenches on the left flank had a swamp behind them. The trenches on the right flank had a flat field behind them, which had something planted before, now all burnt. Yeah...

"I don't like this position," I said.

"Me neither," said Cheetah.

Of course! It wasn't just the position here. There were also the porcupines. There were about a hundred of them, at least, wandering around their position like it was a marketplace. Some had gathered in circles, lighting fires. Others just stood there with their hands in their sleeves. Last of them were wandering around.

Rifles were lying around near the trenches, machine guns were standing aimlessly with their barrels pointed

up at the low sky. In the middle of the road, stuck in the mud up to the hubs, completely out of place, was a rocket launcher. An elderly porcupine sat on the carriage — maybe he was a sentry or just resting after wandering around. Either way, he was harmless: he just sat there, picking his ear with a twig.

The whole scene made me feel sour. If it were up to me, I would have mowed down this entire marketplace with a machine gun... I looked at Cheetah hopefully, but he remained silent, just moving his hooked nose left to right and right to left.

Angry voices sounded behind us, and I turned around. Under the stairs of the last house, two porcupines were arguing. They were fighting over a wooden trough - each one pulling it towards themselves, each spewing vile curses. These two, I would have mowed down with particular pleasure. Cheetah said to me:

"Bring them."

I quickly jumped over to these idiots, smacked one on the hands with my rifle barrel, then the other. When they looked at me, dropping their trough, I jerked my head towards Cheetah. They didn't even squeak. It hit them both suddenly, like in a steam bath. Wiping their faces with their sleeves as they went, they jogged over to Cheetah and stood two steps in front of him, looking like untidy, sweaty piles. Cheetah slowly raised his cane, aimed it like he was playing billiards, and whacked them - once across the face of one and then the other. Then he looked at them, the brutes, and simply said:

"Bring the commander to me. Quickly."

No, guys. Cheetah clearly hadn't expected things to be this bad here. Of course, we weren't expecting anything good. When they send the fighting cats to plug a breach, it's obvious things are dire. But this... even the tip of Cheetah's nose had turned white.

Finally, their commander appeared. A tall, sleepy-looking stick of a man with gray sideburns emerged from behind the houses, buttoning his tunic as he came. He was at least fifty years old. His nose was red and veiny, and he wore smudged pince-nez like the staff officers did in that war. Wet crumbs of chewing tobacco clung to his long chin. He introduced himself as a staff major and tried to switch to a casual tone with Cheetah.

No way! Cheetah gave him such a frosty reception that he seemed to shrink in stature. He had been half a head taller at first, but after a minute, snake's milk! He was already looking up at Cheetah, a little gray old man of average height.

So, it turned out like this. The staff major didn't know where the enemy was or how many of them there were. His task was to hold the village until reinforcements arrived. His fighting force consisted of 116 soldiers with eight machine guns and two rocket launchers; almost all the soldiers were only partially fit for duty, and after yesterday's forced march, twenty-seven of them were lying in those houses over there — some with chafing, some with hernias, and various other issues.

"Listen," Cheetah suddenly said. "What's going on over there?"

The staff major stopped mid-sentence and looked where the polished cane was pointing. My goodness, what eyes our Cheetah has! Only now did I notice: in the biggest circle around one of the fires, amidst the gray jackets of our porcupines, were the disgusting striped jumpsuits of imperial armored infantry. Snake's milk! One, two, three... four rat-eaters by our fire, and these pigs are practically hugging them. Smoking. And even laughing about something...

"Over there?" the staff major said, looking at Cheetah with his rabbit-like eyes. "Are you referring to the prisoners, sir?"

Cheetah didn't respond. The staff porcupine put his pince-nez back on and launched into an explanation. These, you see, are prisoners, but they have nothing to do with us, you see. They were captured in yesterday's battle by the jaegers. Lacking transportation and sufficient personnel for proper guarding...

"Gug" Cheetah said. "Take them to Tick. Just make him interrogate first..."

I snapped the bolt and walked over to the fire. The bastards were smoking and sipping something from mugs. Their faces were content and shiny. Disgusting... and that blond one, patting a porcupine on the back, and the porcupine, that brainless log, was grinning and nodding his head. Are they drunk or something?

I walked right up to them. The porcupines noticed me from a distance, went silent, and started quietly scattering in all directions. Some of them, apparently too scared to move, just sat there, wide-eyed and gaping. The striped ones, though — they turned pale gray. Rat-eaters knew our emblem, had heard of us!

I ordered them to stand up. They stood, reluctantly. I ordered them to line up. They did, having no other choice. The blond one started babbling something in our language — I poked him in the ribs with the barrel, and he fell silent. So, I marched them off, single file, heads down, hands behind their backs. Rats. And they even smelled ratlike... two were strong men, broad-shouldered, and two looked like they were from the latest conscription, scrawny gomers, maybe just a bit older than me.

I hate prisoners. What kind of slime are they, going to war and ending up captured? I get it, they're rat-eaters, but still, it's disgusting, no matter how you look at it... well, here we go: one of the scrawny ones doubled over and started vomiting. Move it, move it, snake's milk! The second one started too. Ugh! How these rats sense impending death, just like real rats. And now, they'd do anything to save themselves — betray, sell out, become slaves...

"Double-time, march!" I barked in their language.

They started running. Slowly, poorly. That blond one was limping. Severely injured, I suppose; twisted his ankle in the latrine. No matter, you'll hobble along.

We reached the edge of the village, where the trucks were parked—the guys saw us and started shouting and whistling. I found a large puddle, shoved the prisoners face-first into the mud, and headed to the front truck where Tick was. And there he was, jumping out to meet me — his face cheerful, the little mustache under his nose bristling, and a bone cigarette holder clenched in his teeth, just like the senior class fashion.

"Well, what do you have to say, brother-doomsayer?" he says to me.

I reported to him: such and such, this is the situation, and the prisoners must be interrogated first. And then I added on my own:

"Don't forget about me, Tick," I said. "After all, I brought them here..."

"You're talking about the collar?" he asks absently, while looking around.

"Exactly! Who brought them here, after all?"

"I just don't see what we can use. We're not taking them all the way to the forest..."

"How about to use the stilts?"

"It's possible, of course, to use the stilts... but why?" He looked at me. "What if we do it without the stilts? Would you take it on?"

Well, there it is. Just as I thought. I'm always unlucky. Is it my fault that my wingman was kept at the headquarters? And how can I do it alone? I won't have the strength. I'll be struggling until evening and then scrubbing myself clean all night.

"You know," I said to Tick. "I don't have a wingman."

"How about alone?" he asks. "Got your cord with you?"

I got carried away with excitement here.

"Will you hold them?" I ask.

He looked at me, and my heart sank immediately.

"Kitten..." he says. "You're going to have fun here while Cheetah is alone there? Take three pairs and get to Cheetah! Quickly!"

There was nothing to be done. It wasn't meant to be, just unlucky. I took one last look at my striped ones, slung my rifle over my shoulder, and shouted at the top of my lungs:

"First, second, third pairs — to me!"

The kittens tumbled off the truck like peas: Hare with Rooster, Nosy with Crocodile, Sniper with that one... what's his name... I wasn't used to him yet, he had just been transferred to us from the Piggan school — he killed the wrong person there, so they sent him to us.

I had noticed it a long time ago but never mentioned it to anyone: if cat happened to off a civilian in a fit of anger, an order would immediately go out. So-and-so with such-and-such a nickname is to be executed for committing a criminal offense. They would drag him out to the parade ground, stand him before his best friends, fire a volley at him, toss his body into a truck for a dishonorable burial, and then you'd hear that the guys had seen him either on an operation or in another unit... and I think that's the right way to do it.

Well, I commanded "double-time," and we hurried back to Cheetah. And Xheetah wasn't wasting any time there. I saw that pole of a man, the staff major, trotting towards us, and behind him was a column of about fifty porcupines with shovels and mattocks, their boots thudding, sweaty with steam rising off them. This meant Cheetah had them digging a new position, a real one, for us. Under the house opposite the medical unit, I saw shovels flashing, and there was a rocket launcher set up, and the whole village was bustling like the

main street on a name day. The porcupines were scurrying about, and not a single one was empty-handed: either carrying weapons, but those were few, and most were dragging crates of ammunition and machine gun mounts.

Cheetah saw us and expressed his satisfaction. He immediately sent Hare's and Sniper's pairs into the jungle for advanced scouting, kept Nosy and Crocodile with him for communication, and said to me:

"Gug. You're the best rocket launcher operator in the unit, and I'm counting on you. See those cockroaches? They're yours. Set up the rocket launcher on that edge, choose a position roughly where our trucks are now. Camouflage well, and open fire when I set the village ablaze. Move out, cat."

When I heard all that, I didn't just run, I practically flew to my cockroaches. My cockroaches, along with the rocket launcher, were stuck in a muddy pothole in the middle of the road and looked like they intended to spend the entire war there. They were barely moving their paws, those lazybones. So, I gave one a smack, kicked another, and hit the third with the butt of my rifle between the shoulder blades, shouting so loudly that it rang in my own ears — my cockroaches started working properly, almost like humans. They lifted the rocket launcher out of the pothole by hand and — march, march — rolled it down the road, with wheels screeching and mud flying, and — straight into another pothole. This time, I had to pitch in myself. You see, guys, you can make porcupines work too, you just need to know how.

So, my situation was like this. I had already chosen a position - I remembered some thick ginger bushes near the trucks and a flat lowland behind them where we could easily dig in, hidden from any devils coming from the jungle. From there, I would see everything: the road up to the jungle, the whole village outskirts if they pushed through the houses, and the swamp on the left if the armored infantry tried to come through there... I also thought I should remember to ask Tick for a few pairs to cover that side. I had twenty rockets in the trays, provided those clerks hadn't thrown any out to lighten their load on the way here... well, we'd check that out soon enough, and in any case, once we dug in, I'd have to send the cockroaches for more supplies. I hate having to ration during a fight. That's not a fight, I don't know what that is... we had enough time until dusk, and when they attacked at twilight, that wild village would light up, and I'd have them all in my sights — shoot at will. You won't regret counting on me, Cheetah!

I finished this last thought almost automatically while lying on my back, and in the gray sky above me, burning fragments were flying like strange birds. I hadn't heard a shot or an explosion, and now I couldn't hear anything at all. I was deaf. I don't know how much time had passed, and then I sat up.

From the jungle, armored vehicles were crawling out four abreast, spitting fire and spreading out into a semicircle battle formation, with another four following behind them. The village was burning. Smoke hung over the trenches ahead, and there wasn't a soul in sight. The field kitchen next to the trading post

was overturned, with its contents spilled out in a brown mess, steaming. My rocket launcher was also overturned, and the cockroaches were lying in a heap in the ditch. In short, I had taken up a perfect position, snake's milk!

Then we were hit by a second barrage. It knocked me into the ditch, flipping me head over heels, filling my mouth with clay and my eyes with dirt. I had just gotten to my feet when the third barrage came. And it kept coming, and coming...

We managed to get the rocket launcher back on its wheels, rolled it into the ditch, and I burned one of the armored vehicles. There were only two cockroaches left now; where the third one went, I had no idea.

Then, suddenly and without transition, I found myself on the road. Ahead of me was a whole bunch of striped ones — close, very close, right next to me. The fire reflected blood-red off their blades. A machine gun was roaring deafeningly next to my ear, I had a knife in my hand, and someone was twitching at my feet, knocking against my knees...

Then, meticulously, as if on the training ground, I aimed the rocket launcher at the steel shield advancing on me through the smoke. I could almost hear the instructor's command: "At the armored infantry... with armor-piercing rounds..." but I couldn't press the trigger because I had a knife in my hand again...

Then suddenly, there was a lull. It was already twilight. It turned out that my rocket launcher was intact, and so was I. Around me gathered a bunch of porcupines, about ten people. They were all smoking, and someone handed me a flask. Who? Hare? I don't

know... I remember that against the backdrop of a burning house about thirty steps away, there was a strange figure: everyone else was sitting or lying down, but this one was standing, and it looked like he was black and naked... he wasn't wearing any clothes — no coat, no jacket. Or was he not naked after all? "Hare, who is that standing there?" - "I don't know, I'm not Hare." - "Where's Hare?" - "I don't know, just drink, drink..."

Then we were digging, hurrying with all our might. This was already a different place. The village was now not to the side but ahead of us. Well, there wasn't much of a village left — just piles of charred wood, but there were burning armored vehicles on the road. A lot of them. Several. The swampy ground squelched underfoot... "I commend you, well done, cat..." — "Sorry, Cheetah, I'm not thinking straight. Where are all our guys? Why are there only porcupines here?" — "Everything's fine, Gug, keep working, keep working, brave brother, everyone is safe, everyone is impressed with you..."

...aha! Got it! Right in the blunt snout. It's backing up, settling on its rear, throwing a shower of sparks into the black sky. They're running, running! "Cat, on the right! On the right! Now!" I don't see anything on the right, and I'm not looking. I turn the barrel that way, and suddenly a shower of liquid fire comes straight at me from the black-red haze. Everything ignites at once — the corpses, the ground, the rocket launcher. And some bushes. And me. It hurts. An infernal pain. Like Baron Tragg...

Puddle, I need puddle! There was a puddle here! They were lying in it! I put them there, snake's milk, but I should have put them in the fire, in the fire! No puddle... the ground was burning, the ground was smoking, and suddenly someone with inhuman strength knocked it out from under my feet...